Mouseless is a productivity / utility app that allows users to efficiently control the mouse with just the keyboard.

How to use these docs

Navigation

The **sidebar** on the left lists all chapters. Click the **hamburger menu (☰)** to show or hide it.

To navigate to the next or previous chapter, use the **arrow buttons** at the side or bottom of the page or the `Left` and `Right` arrow keys on the keyboard.

Top menu bar

Icon	Action
☰	Toggles the sidebar.
🖌	Opens the color theme picker.
🔍	Opens the search bar.
🖶	Prints the entire book.

Tapping the menu bar will scroll the page to the top.

Search

Press the `S` key (or click the search icon (🔍) in the menu bar), to open the search bar.

Results are shown in real time. The `Up` and `Down` arrow keys can be used to navigate the results, and `Enter` will open the highlighted section.

Matched search terms will be highlighted in the text. Clicking a highlighted word or pressing the `Escape` key will remove the highlighting.

Single-page view / AI-assisted help

To view the docs in a single page, click the print icon in the menu bar, then cancel the print operation.

The content can then be copied and pasted into any AI chat for interactive Q&A about Mouseless.

Getting started

MacOS

To install:

- Download the .dmg file from https://mouseless.click
- Open the .dmg file
- Drag `Mouseless.app` into the `Applications` folder

You can also install in the terminal via **Homebrew**:

- `brew install --cask mouseless` (stable channel)
- `brew install --cask mouseless@preview` (prerelease channel)

First-time usage:

- Open Mouseless (e.g. via spotlight search)
- Read/agree to the terms of service
- Start trial or activate
- Give Mouseless the `Accessibility` permission when prompted
- Mouseless will **restart itself** once the permission is granted (if it doesn't, restart it manually)
- Enjoy!

To update:

As of v1.0.0 Mouseless has in-place updating – just choose 'Install update' when one is available.

To run Mouseless on startup:

Mouseless does not (yet) add itself as a 'Login Item', so you must do this:

1. Open 'Login Items' via spotlight search
2. Click the '+' button
3. Add Mouseless.app

Windows

To install (or update):

- Download the latest Windows installer (.exe) or portable (.zip) file from https://mouseless.click
- Run the installer (or extract the .zip)
- If installing, choose whether to install for current user (no admin privileges required) or for all users (admin privileges required)

First time usage:

- Open Mouseless, e.g. via the start menu if installed, or if using the portable, by running the `mouseless\mouseless.exe` file in the extracted folder/location
- Read/agree to the terms of service
- Start trial or activate
- Enjoy!

Linux

Wayland users: Please see the Enabling keyboard/mouse access on Wayland page before running Mouseless. If you're unsure if you're on Wayland or X11, run `echo $XDG_SESSION_TYPE` in a terminal.

Install

On Linux, Mouseless uses `flatpak`, a package manager that allows apps to have self-contained dependencies (either in the app itself or in standardized runtimes) for cross-distro compatibility.

The Mouseless developer(s) host their own flatpak repo, which means releases/updates are GPG-signed to verify their authenticity. (GPG key, fingerprint: `C739 2023 E17C 0B73 1291  668E 1D97 183A C9AF 10C4`)

There is also a standalone bundle, but it requires manual updates (download latest bundle and re-install).

Recommended install method (flatpak repo):

```
# Ensure Flathub repo present so Gnome runtime is available
flatpak remote-add --user --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo

# Add GPG-signed Mouseless repo
flatpak remote-add --user --if-not-exists sonuscape \
  https://dl.sonuscape.net/flatpak/sonuscape.flatpakrepo

# Install Mouseless (stable channel)
flatpak install --user net.sonuscape.mouseless

# Run
flatpak run net.sonuscape.mouseless
```

- Uses the GPG-signed `dl.sonuscape.net` repo
- Allows for easy updates via `flatpak update` / software managers
- Ensures Mouseless displays correctly in software manager apps
- Enables other (future) Sonuscape software to show up in searches and be easily installable
- For the **preview channel**, use `flatpak install --user net.sonuscape.mouseless//preview`

Method B (.flatpakref file):

```
# Ensure Flathub repo present so Gnome runtime is available
flatpak remote-add --user --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo

# Install Mouseless
flatpak install --user \
  https://dl.sonuscape.net/flatpak/net.sonuscape.mouseless.flatpakref

# Run
flatpak run net.sonuscape.mouseless
```

- Uses the GPG-signed `dl.sonuscape.net` repo
- Allows for easy updates via `flatpak update` / software managers
- Compared to the full repo method (above):
  - May have degraded UX in software managers
  - Does not enable other Sonuscape software to be findable in your software manager
- For the **preview channel**, use `flatpak install --user https://dl.sonuscape.net/flatpak/net.sonuscape.mouseless.preview.flatpakref`

Method C (standalone .flatpak file):

```
# Ensure Flathub repo present so Gnome runtime is available
flatpak remote-add --user --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo

# Download latest bundle and .sha256sum
curl -LO
https://dl.sonuscape.net/flatpak/bundle/mouseless/latest/Mouseless.flatpak
curl -LO
https://dl.sonuscape.net/flatpak/bundle/mouseless/latest/Mouseless.flatpak.sha256s
um

# Verify download, then install only if it passes
sha256sum -c Mouseless.flatpak.sha256sum && \
  flatpak install --user ./Mouseless.flatpak

# Run
flatpak run net.sonuscape.mouseless
```

- Does not use the repo (self-contained file)
- Updates must be performed manually (download latest and re-install)
  - In-app update notifications are still available (if not disabled)
- Index of available bundles
- For the **preview channel**, change the filenames in the commands to `Mouseless.preview.flatpak`

Notes:

- All three methods will install the official GNOME runtime (~400MB) if not already present on your system.
  - If you don't already have the Flathub remote, the installer offers to add it; confirm with `y`
- Software manager integration: After installing with the commands above, Mouseless should appear in your default software manager GUI (e.g. GNOME Software). You can also install by downloading the sonuscape.flatpakrepo, double clicking it to open in your software manager, then search for/install Mouseless and run it from there or your system launcher. **Note:** the software manager may only display the stable branch.

Installation troubleshooting

If the above steps did not automatically instally the GNOME runtime, run this command, then try your chosen install command again:

```
flatpak remote-add --user --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install --user flathub org.gnome.Platform//50
```

On older distros: you may see errors like `URI .../summary exceeded maximum size` (seen on Ubuntu 20 w/ flatpak 1.6). To fix this, you need a newer version of flatpak than may be available from your default repository:

```
sudo add-apt-repository ppa:flatpak/stable   # maintained by Flatpak team
sudo apt update && sudo apt install flatpak
```

Switching between stable and preview

To switch an existing install to the preview channel:

```
# Add the Mouseless repo (needed if you installed from a .flatpakref file)
flatpak remote-add --user --if-not-exists sonuscape \
  https://dl.sonuscape.net/flatpak/sonuscape.flatpakrepo

# Install the preview branch, and make it the one that runs
flatpak install --user net.sonuscape.mouseless//preview
flatpak make-current --user net.sonuscape.mouseless preview
```

To switch back to stable:

```
flatpak make-current --user net.sonuscape.mouseless stable
```

Uninstall

Mouseless: `flatpak uninstall net.sonuscape.mouseless`

Repo: `flatpak remote-delete sonuscape`

Pinning a specific version (prevent updates)

Current version:

```
flatpak mask net.sonuscape.mouseless
```

Any version:

```
# List your remotes (to find <remote> for the next command)
flatpak remotes

# View commit (version) history
#  - <remote> is the name from `flatpak remotes` above
#  - it's `sonuscape` if you installed the .repo file
flatpak remote-info --log <remote> net.sonuscape.mouseless

# Install specific version
flatpak update --commit=<GOOD_COMMIT_HASH> net.sonuscape.mouseless

# Prevent future updates
flatpak mask net.sonuscape.mouseless
```

In-app update notifications will still be shown (unless disabled in app settings).

To resume normal updates (unpin):

```
# Remove mask
flatpak mask --remove net.sonuscape.mouseless
# Update app
flatpak update net.sonuscape.mouseless
```

Troubleshooting

Not working right? See the troubleshooting section for solutions and workarounds.

Using Mouseless

Note: these docs currently show the default keybindings. If a command is unassigned by default, the command name is used instead.

To view/edit the keybindings and other options, press `Tab` (`edit config` command) while the overlay is showing, **OR** choose `Settings...` from the Mouseless menu in the status bar (Mac) / system tray (Windows) / window header bar (Linux). You can use `ctrl+F` / `cmd+F` to find a setting in the config editor.

Clicking, moving, and dragging

With free mode

Free mode allows for relative movement, with no overlay needed to move, click, or drag.

Activating / deactivating

To **activate free mode**, use the `enter free mode` command or `toggle free mode` command (`OptionLeft tap` on Mac, `ControlLeft tap` on Windows and Linux).

To **exit free mode**, use the `exit free mode` or `toggle free mode` command.

There is also a `hold for free mode` command (beta): free mode stays active only while the assigned key is held.

Auto-off

By default, free mode **turns off automatically** after 10 seconds without usage (adjustable via the `Free mode auto off` option; set to `0` to disable).

In 1.1 preview: the new default is `0` (disabled).

Actions in free mode

While free mode is on:

- to **move**, use `I`, `K`, `J`, and `L`

- to **click or click+drag**, use `Space` (left mouse button), `R` (right), `E` (middle), `Q` (back), and `W` (forward) as if they were mouse buttons
- to **scroll**, use `M` and `,` wheel up/down, and `.` and `/` for left/right
- to **increase movement or scroll speed**, use `S`, `D`, and `F` – the more of them you hold, the greater the increase
- to **decrease movement or scroll speed**, hold `A`

With the overlay

Showing/hiding the overlay

- To **show the overlay**, `tap` the `CommandLeft` (`ShiftLeft` on Windows and Linux) key (`show overlay` command)
  - **Note:** this is a `tap` command by default, so you must press and release the key relatively quickly for it to register (< 0.2 seconds by default, see Tap vs keydown for more info)
- To **hide the overlay** press the `Escape` key (`hide overlay` command)

Actions in overlay mode

- To **click**:
  - ***Choose a cell***: while the overlay is showing, type the two characters of any cell, then:
    - ***Click at cell center***: press `Space` to click at the center of the cell, where the virtual cursor is (`execute mouse action` command)
    - ***OR Click at a sub-cell***: press any character you see in the subgrid to click at its sub-cell
  - ***Click at virtual cursor***: you can press `Space` any time during cell selection to click where the virtual cursor is (or before selection if `Initial action location` is set to `screen_center`)
  - ***Click at system cursor***: if `Initial action location` is set to `system_cursor`, press `Space` before selecting a cell
- **To undo cell selection**: if you press the wrong key while selecting a cell, press `Backspace` to go back one level (`undo last key` command)
- To **right click**, do any of the following:
  - execute the `right click` command
  - use the `hold for right button` command: hold the `Shift` key (`CommandRight` on Mac) key while pressing the final key of a click action

    - use the `cycle mouse button` command to choose `right`, then execute a mouse action

- To **use other mouse buttons**, use the appropriate `hold for` command, or the `cycle mouse button` command, or `middle/back/forward click` commands in the same way as described above for right clicking

- To **double-click or triple-click**, either:

    - press the final key of a click action multiple times within the `multi-click threshold`
    - use the `cycle click count` command

- To **click-and-drag / drag-and-drop**, use the `hold for drag` command, i.e.:

    - to **begin the drag**: hold the `AltLeft` key (`CommandLeft` on Mac) key while pressing the `execute mouse action` hotkey or a subgrid key
        - the overlay will remain up for you to choose the point you want to drag to or drop at
        - to **drag to a point** (without releasing the drag), hold the `AltLeft` key (`CommandLeft` on Mac) on subsequent `execute mouse action` / subgrid key presses
        - to **cancel a drag** (release at the system cursor), press `Escape` (`release hold/drag` command)
    - to **release/drop**: enter in a click-coordinate as normal, without holding the `hold for drag` hotkey

- **Click-and-drag alternative:** you can also use the `cycle mouse action` command to set the action to be performed. During a drag, it cycles between `drag` and `drop`.

- To **move the mouse cursor**:

    - tap `AltLeft` (`OptionLeft` on Mac) while the overlay is up (`execute mouse move` command)
    - hold the key assigned to the `hold for move` command during the final keypress of a a mouse action
    - use the `cycle_mouse_action` command to set the action to `move`

- To **repeat the last action executed**, use the `repeat last mouse action` command

Extra precision: Cursor nudges

If you need to click or move to small targets in between subgrid points, you can use 'nudges' as follows.

1. Ensure the `hold subgrid key for nudge` setting is ON in each grid config in the `Grid Options` section of the config editor.
2. Ensure the `Subgrid nudge` commands are assigned keys in the `Keymap` section. The default values give each command both a left-hand keybinding and a right-hand keybinding (assuming a QWERTY layout), so that nudges can be executed regardless of which subgrid key is held.
3. To execute a nudge:

- Activate the overlay, and select a cell to bring up the subgrid.
- Instead of tapping a subgrid key to immediately execute a click, *hold* the subgrid key nearest to your target.
- While this key is held, use the `subgrid nudge` hotkeys to move the cursor.
- When the cursor is in the desired location, simply release the subgrid key you've been holding to execute the click.
    - OR press the hotkey for `right/middle/back/forward click` to execute a click with that button.

4. The nudge size / precision can be adjusted with the `nudges per cell` setting in the `Grid Options` section.

If you use **QMK** or similar keyboard firmware and use dual-use hold/tap keys (and have those keys in your subgrid), it's recommended you leave `hold subgrid key for nudge` OFF, as it will not behave as desired when those keys are held.

With label mode (beta)

In 1.1 preview: label mode puts a label on each clickable element in the frontmost app; type a label to click its element. It works on macOS, Windows, and Linux on GNOME or an X11 session; it doesn't work yet on KDE, sway, Hyprland, niri, or other Wayland compositors. On GNOME, log out and back in once after installing Mouseless, so its GNOME Shell extension loads.

Showing/hiding labels

- To **show labels**, `tap shift+CommandLeft` on Mac (`shift+AltLeft` on Windows, `shift+ctrl` on Linux) (`toggle label mode` command)
- To **hide labels** without clicking, press `Escape` , or use `toggle label mode` again

- To **undo a typed character**, press `Backspace`

Actions in label mode

Typing a label left-clicks its element. The overlay's `hold for` and `cycle` commands change the click as they do for `execute mouse action`, as they stand when you type the label's last character. And unbound modifier can be applied to clicks if held while typing the last character.

Alternate workflow: quick select + free mode

The built-in `quick select + free mode` preset (in the `Mouse actions` subsection of the `Behavior` section) lets you choose a screen location coarsely with the overlay, then use free mode for finer movement before clicking or dragging. It sets:

- `Grid action level`: `2`
- `Default action`: `move`
- `Free mode after move`: `true`

There's an accompanying `standard` preset (`subgrid`, `click`, `false`) to restore the default workflow.

Global mouse buttons

These commands execute mouse button presses without the overlay, typically with a mouse or other pointing device still controlling cursor movement.

Use cases include:

- clicking while maintaining precise cursor position/movement
- preventing/alleviating repetitive strain injury
- user preference over other clicking mechanisms

To use these commands, in the `Keybindings -> Global mouse buttons` section of the config editor, assign a key or key combo that won't interfere with your regular keyboard usage. All five mouse buttons (left, right, middle, back, and forward) can be assigned to. After assigning, you should then be able to use keyboard keys as mouse buttons, without having to bring up the overlay or activate free mode.

Modifier clicks (or other actions)

Modifier clicks are available in overlay mode and free mode (**in 1.1 preview:** also in label mode, where `shift` doesn't count).

If a modifier key is held during a mouse action, and *isn't* assigned to a `hold for` command, a native mod+mouse event will be simulated. In other words, to execute a shift+click (e.g. to select a group of items in file explorer):

- Select a cell in the overlay, or position the mouse cursor in free mode
- Hold `shift`
- Execute the mouse action (e.g. by pressing `Space` or a subgrid char)
- *Then* release `shift`

Wheel / scrolling

To use the mouse wheel, `tap` the `OptionLeft` key on Mac or `ControlLeft` key on Windows and Linux (`toggle free mode` command) to enter and exit free mode.

While in free mode, use:

- `M`, `,`, `.`, and `/` to scroll up/down/left/right
- `S`, `D`, and `F` to increase the speed (the more pressed, the greater the increase)
- `A` to decrease the speed

There are also `fast`, `step` (precise increment, responds to autorepeat when held), and `step large` commands, and on Mac and Windows there are `jump to top/bottom/left/right` to jump to the edge of a scrollable area.

To adjust scrolling speed / step sizes, as well as `auto off` duration, see the `Free mode` subsection of the `Behavior` section in the config editor.

See here for the rest of the free mode controls / capabilities.

Multiple monitor usage

- To **move the overlay between monitors**, `tap` the `ShiftLeft` or `ShiftRight` keys while the overlay is visible (`switch monitor left/right` commands; monitors are ordered by their x position).

- To **assign different grid configs to different monitors**:
    - In the `Grid Options` section of the config editor, use the `+` and `-` buttons to add/remove grid configs
    - With `monitor assignment mode` set to `auto`, the first grid config will be assigned to monitors with a horizontal aspect ratio, and the second will be assigned to monitors with a vertical aspect ratio
    - For more customized assignment, set `monitor assignment mode` to `custom`, and use the `custom monitor assignments` field to assign grid_config names to specific monitors

Menu/gui actions

- To **open the config editor**, press the `Tab` key while the overlay is up (`edit config` command)
- To **close dialogs, tooltips, etc**, press the the `Escape` key (`close ui element` command)

Customizing Mouseless

The config editor

- To **open the config editor**, press `Tab` while the overlay is visible, **OR** choose `Settings...` from the Mouseless menu in the status bar (Mac) / system tray (Windows) / window header bar (Linux),
- Many **tooltips** are available for in-context guidance. To see them, hover over:
  - The label on many fields
  - Some built-in presets
  - The info icons next to some sections
  - The info icon at the bottom-left of the config editor
- Changes you make are applied automatically
- The config editor is **searchable** via `ctrl+F` (`cmd+F` on Mac)
- To **save**, hit the save button (or press `ctrl+S` / `cmd+S`)
- To **save/load presets, or restore saved or defaults** use the kebab menus in the section/subsection headers
  - To **update or delete a user preset**, right-click it to show a context menu
- To **restored saved or default values** for a single field, right-click the field to show a context menu
- Use the kebab menu at bottom to **import a config file, export a config file, or restore defaults for the entire config**

Field editing tips

- You can press `Enter` in a text field to make it validate / take effect. Leaving the text field with `Tab` or the mouse also works.
- Colors can be copied and pasted (with `ctrl+C` and `ctrl+V` (`command` on Mac)) while focused, or by dragging and dropping from one to another. The copied format is hex (e.g. `#4e6c95`).

Manually managing config files

- You can choose `Open config folder` from the Mouseless menu, and edit/duplicate/delete files at will. `config.yaml` is the one loaded at app startup.
    - **NOTE:** if the `config.yaml` file is removed, or modified in such a way that it has invalid syntax, the default config will be used the next time the app is started.
- The location of the config file is:
    - Mac:

```
~/Library/Application Support/Mouseless/configs/config.yaml
```

    - Windows:

```
~/AppData/Local/Mouseless/configs/config.yaml
```

    - Linux:

```
~/.var/app/net.sonuscape.mouseless/data/mouseless/configs/config.yaml
```

NOTE: the information for each field that follows is largely the same as is found in the in-app tooltips

General options

- **Automatically check for updates**: When on, Mouseless checks for new versions daily and at startup.
- **Check for preview releases**: When on, update checks include preview (prerelease) versions.
- **Hide welcome screen**: Skips the welcome/tutorial window at startup.
- **Ignore update to version**: Suppresses update notifications for the given version.

Keyboard options

- **Keyboard layout** (Grid section; **in 1.1 preview:** `Behavior` page): The keyboard layout used to interpret keys for the overlay and hotkeys, regardless of the current system layout.
- **Excluded input devices** (Wayland; Grid section; **in 1.1 preview:** `Overlay Grid` page): Input devices Mouseless should never take control of. Matches by case-insensitive name substring, or by id (e.g. `ids:046d:c52b`; a `0` on either side acts as a wildcard). Run Mouseless with `--list-input-devices` to view device names/ids.

Grid (overlay) options

The dimensions and content of an overlay is determined by a 'grid config'.

- To add or remove a grid config, use the `+` / `-` buttons under the lowest grid config.

Grid config fields

- **Level _ columns**: The number of cells across the X dimension at this level.
- **Level _ rows**: The number of cells along the Y dimension at this level
- **Level _ keys**: The keys you press to select a cell at this level. They wrap to multiple rows if necessary. **Note:** If left blank for level 2 or subgrid, the keys from level 1 will be used.
- **Always show subgrid**: Shows a subgrid in every cell, all the time, which can make it easier to know ahead of time which sub-cell you'll want to click. This comes at the cost of more visual noise, however. Currently only available when `grid line style` is set to `lines`.
- **Hold Subgrid Key For Nudge**: When on, holding a subgrid key allows you to use the `subgrid nudge` commands to move the cursor, then release the subgrid key to execute the action.
- **Nudges per cell**: Determines nudge size. I.e. if set to `4`, then 4 nudges will move the cursor to the same point in next subgrid cell.

Monitor assignment fields

- **Monitor assignment mode**: Controls how grid configs are assigned to monitors.
  - **auto**: the first grid config gets assigned to horizontal monitors, and the second gets assigned to vertical monitors.
  - **single**: the first grid config gets assigned to all monitors.

    - **custom**: grid configs are mapped by name, via the `custom monitor assignments` field.
- **Custom monitor assignments**: List of grid config names, separated by comma+space (e.g. `A, B`). The first is assigned to the primary monitor, the second to the second monitor, and so on.

Behavior options

Overlay / cursor

- **Initial overlay monitor**: The monitor that the overlay will appear on when shown. (`with_mouse` and `with_keyboard_focus` values not available on Wayland Linux).
- **Initial action location**: Determines where mouse actions are executed (via `execute mouse action`) when no cell is selected.
- **Grid action level**: Determines the grid level that mouse actions get executed at (i.e. how many keypresses it takes). Use the `cycle grid action level` command to change this value while the overlay is up.
    - **subgrid**: When a leaf cell is selected, a subgrid is shown and `Subgrid keys` are used to execute an action.
    - **2** or **1**: When a cell at the given level is selected, a mouse action is executed immediately, at its center.
- **Default action**: The action (`click`, `move`, or `drag`) that `execute mouse action` performs in the default state (i.e. when no `hold for` or `cycle` command modifies it).
- **Mouse value cycle behavior**: What happens to values set with the `cycle button/action/count` commands after a mouse action is executed.
    - **once**: the value returns to its default after executing an action.
    - **persist**: the value remains until cycled again.
- **Free mode after move**: When on, free mode is entered after executing a `move` action from the overlay. See quick select + free mode.
- **Hide cursor on click**: When on, the cursor moves out of the way after a click is executed.
- **Hide location**: The corner or edge of the clicked screen that the cursor will move to. You can choose any edge (top/bottom/left/right) or corner (top_left/top_right/bottom_left/bottom_right).

Movement / timing

- **Move duration (ms)**: The time taken to move the mouse cursor during move and drag actions.
- **Move system cursor with virtual**: When this is on, the system cursor will smoothly move as you select a cell, following the virtual cursor.
- **Multi-click threshold (ms):** For double and triple clicks, this sets the maximum time that can pass between keypresses before it no longer triggers a native double or triple click.
- **Tap threshold (ms)**: For tap-triggered keybindings, this is the maximum time a key can be down before its no longer considered a tap.

Free mode

- **Base move speed (px)**: The amount the mouse cursor moves each tick during `move up/down/left/right` commands.
- **Move speed multiplier**: Movement speed is multiplied by this amount for each `hold for speed increase` hotkey held.
- **Movement easing factor**: Controls the smoothness / acceleration of cursor movement. 0.1 = loose, 0.2 = natural, 0.3 = snappy, 0.9 = instant.
- **Base wheel speed**: The amount scrolled each tick during `wheel up/down/left/right` commands. Measured in pixels on Mac, lines on Windows.
- **Wheel speed multiplier**: This multiplier gets applied:
  - to the `fast` mouse wheel actions
  - when `hold for speed increase` is active
  - when multiple keys for the same direction are pressed.
- **Wheel easing factor**: Controls the smoothness / acceleration of wheel movement. 0.1 = loose, 0.2 = natural, 0.3 = snappy, 0.9 = instant.
- **Wheel step size**: The amount scrolled each time a `wheel step` command is executed.
- **Wheel step size large**: The amount scrolled each time a `wheel step _ large` command is executed.
- **Free mode auto off (seconds)**: If no actions are done for this many seconds, free mode will turn off. Set to `0` to disable.

Misc

- **Send escape after global alt tap (Windows)**: Simulates a press of the `Escape` key after certain global commands (e.g. show overlay, enter free mode) are triggered by an `AltLeft/Right tap`. Helps to deactivate the focused app's menu bar (which typically activates when `Alt` is tapped). **In 1.1 preview:** applies only when `Alt/Win tap mask key` is empty.

Style / Appearance options

- **Master opacity**: Overall opacity of the overlay. Multiplicative with other opacities – e.g. if `master opacity` is `0.5` and and `background opacity` is `0.8`, the final background opacity will be 0.4 (40%).
- **Color input mode**: The extra text input to show for editing color values. Note that rgb hex values (e.g. `#ff0000`) can always be copied/pasted while the color picker element is focused.

Background

See this youtube video for an explanation / demonstration of the multi-color system.

- **Background colors**: The colors/opacities of the top-level cells, including underneath the highlight.
- **Color block size**: Number of level 1 cells each background color is applied to, before cycling to the next. If set to `0`, the colors are distributed evenly; e.g. if there are 30 top level cells and 3 colors, each color block will be 10 cells across.
- **Use color transforms** Allows you to create variations of the base background colors across top-level cells, as well as create visually distinct regions within top-level cells.
- **BG Color Transforms - Level 1**: Use these to transform the cells **within each color block** by adding or multiplying HSLA values (hue, saturation, lightness, alpha). Use "hsla_add" or "hsla_mult", respectively. This can be convenient for distinguishing columns and/or rows within each color block. Editing the base background color(s) will shift all the transformed colors accordingly.
- **BG Color Transforms - Level 2**: Use these to transform the background colors of **2nd-level cells**. This allows you to create visual blocks within each top-level cell, e.g. to distinguish left hand keys from right hand keys in the subgrid, and/or top/middle/bottom keyboard rows.

Highlight

- **Highlight color**: The color given to the currently active cell(s) of the overlay.
- **Highlight animation (ms)**: The time it takes (in milliseconds) for the highlight to move/scale to its new location and size. Set to `0` for instant movement.
- **Show initial highlight**: When on, the highlight will be shown before any cells are selected, covering the entire overlay. Useful when `background colors` have zero opacity.

Grid

- **Grid line style**: **Note**: Currently, `Always show subgrid` is only available for the `lines` style.
- **Grid line width (px)**: **Note**: Subgrid lines use this value only if `grid line style` is `dots`, otherwise they are assigned `1`.
- **Grid line color**: The color/opacity of the main grid lines.
- **Subgrid line color**: The color/opacity of the subgrid lines.

Text

- **Font**: The font used in the overlay. You can use the `cycle font` command to change this while the overlay is up.
- **Font weight**: Controls the boldness/lightness of the text. The actual effect varies by font.
- **Y offset**: Moves the text up/down, by an amount scaled to the font size. This helps correct positioning for fonts that are not vertically centered.
- **Font size multiplier**: Scales the size of the characters within the main grid.
- **Subgrid font size multiplier**: Scales the size of the characters within the subgrid.
- **Char display**: Controls which characters are shown in each cell.
- **Char anchor**: Whether characters are spaced center-to-center (`center`) or edge-to-edge (`edge`).
- **Char spacing strategy**: Sets how the spacing of characters within a cell is adjusted: `cell_width_relative`, `font_size_relative`, or `evenly_spaced`.
- **Char spacing cell width ratio**: `0` -> characters will be approximately touching edge-to-edge `1` -> characters will be evenly spaced across cells
- **Char spacing font size ratio**: `0` -> characters will be approximately touching edge-to-edge `1` -> characters will be about as far apart as an uppercase character is tall
- **Text color**: The `opacity` value here gets applied to a layer with the text and text shadow combined, so that the shadow doesn't show through the text.
- **Subgrid text color**: "The `opacity` value here gets applied to a layer with the text and text shadow combined, so that the shadow doesn't show through the text.

- **Text shadow rgba**: The `opacity` value here is in addition to what gets applied by the `text color opacity` (e.g. if both are `0.5`, the shadow opacity is effectively 0.25).

Cursor

- **Cursor size**: The size (width and height) of the virtual cursor, in pixels.
- **Cursor color**: The color/opacity of the virtual cursor in its default state.
- **Cursor right button color**: The color/opacity of the virtual cursor when `hold for right button` is active or `cycle button` has been used to set the button to `right`.
- **Cursor move color**: The color/opacity of the virtual cursor when `hold for move` is active or `cycle action` has been used to set the action to `move`.
- **Cursor drag color**: The color/opacity of the virtual cursor when `hold for drag` is active or `cycle action` has been used to set the action to `drag`.
- **Cursor other color**: The color/opacity of the virtual cursor when not in one of the states above.

HUD

- **Toast opacity**: The opacity of the toast/status window, so it doesn't overly obscure underlying screen content.
- **Free mode status display** (Mac): How free mode status is indicated: `toast`, `cursor` (a dot indicator at the cursor; macOS 14+), `both`, or `none`.
- **Free mode indicator color / size / offset X / offset Y** (Mac): The color, size (px), and position offset of the cursor dot indicator.

System / debug options

These are at the bottom of the config editor.

- **Event tap location** (Mac): Where Mouseless inserts its event tap. Changing this may alleviate hotkey conflicts with other apps (see App conflicts).
- **Hud window level** (Mac): The window level of Mouseless' overlay/HUD windows. Adjusting this may fix issues when using software that captures displays (e.g. yabai).
- **Display input**: Shows keyboard input on screen (for demos/troubleshooting). Resets to `none` on app restart.
- **Log level**: Controls how verbose the log file is (`debug`, `info`, `warning`, `error`).
- **Overlay scaling / Mouse scaling / Mouse screen offset scaling** (Linux): Manual corrections for display scaling issues – see Troubleshooting.

- **Debug options**: A JSON object for less-common overrides – see Command line & debug options.

Keybindings / command reference

The `Keymap` section of the config editor allows you to customize program keybindings

A condensed form of much of this info is available in-app in the `Keymap` section's info icon tooltip.

Note: if a key is present in the overlay at the current grid level, as well as a keybinding in the keymap, the overlay will take precedence.

Key names

The key names used/accepted mostly follow web conventions, with some exceptions, and some allowances for aliases.

- Printable non-whitespace characters just use the literal character
  - `A` through `Z`, `0` through `9`, `-=`[]\;',./`
    - **Note:** the non-shifted version of non-alpha characters should be used (e.g. `1` instead of `!`), and the uppercase version of alpha characters should be used (`A` instead of `a`, to avoid ambiguity with modifier shorthands (upcoming feature))
- Whitespace characters are named:
  - `Space`, `Tab`, `Enter`
- Modifier keys
  - Allowed aliases:
    - `Command` / `Cmd` / `Meta` / `Win` / `Gui`
    - `Alt` / `Option` / `Opt`
    - `Control` / `Ctrl`
  - Left/right-specific, or universal versions
    - e.g. `Shift`, `ShiftLeft`, or `ShiftRight`
    - **Note**: side-specific modifier keys are not yet supported as modifiers, only as main keys.
- Other named keys list
  - `ArrowLeft`, `ArrowRight`, `ArrowUp`, `ArrowDown`
  - `Numpad0` through `Numpad9`, `NumpadAdd/Subtract/Multiply/Divide/Decimal/Equal/Comma/Enter`
  - `F1` through `F20`, `Fn`
  - `Backspace`, `Delete`, `Escape`, `PageUp`, `PageDown`, `Home`, `End`, `Insert`

- `NumLock`, `CapsLock`
- `ContextMenu`, `VolumeUp/Down/Mute`
- `Lang1`, `Lang2`, `IntlYen` (¥), `IntlRo` (ろ), `IntlBackslash`

Assigning multiple keybindings to a command

To assign multiple keybindings to a single command, separate them with `, ` (note the space). E.g. `Escape, CommandLeft tap`.

Tap vs keydown

Actions can be triggered either by a keydown event, or by a `tap` – a downstroke and upstroke of the same key within `tap_threshold` milliseconds, uninterrupted by other keys.

To specify a `tap` keybinding, simply add `tap` (with a space) after the keyname, e.g. `CommandLeft tap`. `CommandLeft` on its own as a keybinding will cause the action to be triggered on the key downstroke (which can make interaction a bit snappier).

Double and triple taps

To assign a double or triple tap, use `double tap` or `triple tap` at the end of the hotkey, e.g. `CommandLeft double tap` or `alt+R triple tap`

Using modifiers as...modifiers

To add a modifier to a keybinding in the usual way (like it is used for cmd+C for copy, cmd+S for save, etc), put them before the main keypress with a `+`, e.g.:

- `cmd+Tab`
- `cmd+Tab tap`
- `cmd+alt+shift+ctrl+Tab`

The same aliases (see Key names) are allowed as when the modifier is used as the main key.

Key order

Note that the main key in a keybinding must be the one that is pressed last, and if it's a `tap` key binding, it must also be released first.

E.g. `cmd+Ctrl tap` requires: cmd down -> ctrl down -> ctrl up

QMK and similar firmware often have `hyper key` macros that have a first-in first-out press/release order (e.g. cmd down -> ctrl down -> cmd up -> ctrl up), which does not work with `tap` commands that have a modifier as the main key (though the *non-tap* command should work with the right modifier as the main key).

Avoiding accidental triggers

If a single modifier tap results in accidental triggers during computer usage, you can assign a more deliberate action. Some strategies:

- Add a modifier to the tap, e.g. `shift+CommandLeft tap`
- Require a double tap, such as `ControlLeft double tap`.
- Assign a keybinding to **Toggle app enabled**, then disable Mouseless before games or other situations where you will not use it.

If a tap command still triggers unexpectedly, see App conflicts for possible causes and `tap_threshold` advice.

Command reference

Note: This information is largely the same as the in-app tooltips when you hover over the command name.

Overlay

- **Show overlay**: Shows the overlay.
- **Hide overlay**: Hide the overlay, without executing any mouse action.
- **Toggle overlay**: Shows or hides the overlay.
- **Switch monitor left**: Moves the overlay to the next monitor to the left (by x position).
- **Switch monitor right**: Moves the overlay to the next monitor to the right (by x position).
- **Increase overlay opacity**: Increases the `master opacity` value.
- **Decrease overlay opacity**: Decreases the `master opacity` value.

- **Invert colors**: Inverts all the colors on the overlay, which can help with visibility at times. Execute it again to toggle back.
- **Toggle overlay lock (beta)**: When locked, the overlay will not respond to keypresses, and keypresses will pass through to focused app. May be useful while editing the config. **This is a beta feature**, and not fully implemented – some commands will still affect the overlay.
- **Toggle continuous mode**: When continuous mode is on, the overlay stays up after executing a mouse action.
- **Toggle continuous mode until closed**: Continuous mode will be turned off after the overlay is hidden.
- **Cycle grid action level**: Cycles the value of the `Grid action level` field.
- **Cycle font / cycle font reverse**: Cycles the value of the `Font` field, forward or backward.

Mouse actions

- **Execute mouse action**: In the default state, executes a left click at the virtual cursor. Can execute other actions/buttons/click_counts if `hold for` or `cycle` commands are used. **Double and triple clicks** can be executed by pressing multiple times.
- **Execute mouse move**: Moves the system cursor to the virtual cursor.
- **Undo last key**: Undoes the last key when selecting a cell.
- **Release hold/drag**: During a drag action, 'drops' at the system cursor.
- **Repeat last mouse action**: Repeats the last mouse action, at the same location it was executed at.
- **Hold for move**: When held, `execute mouse action` will move the system cursor to the virtual cursor.
- **Hold for drag**: When held, `execute mouse action` will start (or continue) a drag operation (i.e. moving the mouse while mouse button is held down).
- **Hold for right/middle/back/forward button**: When held, `execute mouse action` will execute a right/middle/back/forward click at the virtual cursor.
- **Cycle mouse action type**: Sets the action (click, move, drag) that `execute mouse action` will trigger.
- **Cycle mouse button**: Sets the mouse button (left, right, middle, back, forward) that `execute mouse action` will use.
- **Cycle click count**: Sets the click count (single click, double click, triple click) that `execute mouse action` will trigger.
- **Left/right/middle/back/forward click**: Executes a click with the given button at the virtual cursor, regardless of any cycled or held button.
    - **Note**: whether cycled values persist after an action is controlled by the `Mouse value cycle behavior` setting.

Subgrid nudge

- **Subgrid nudge up/down/left/right**: These commands move the cursor when `hold subgrid key for nudge` is on and a subgrid key is held.
    - **Note**: If you use keys that are also present in the subgrid, you should give each command a key for the left hand and the right hand.

Dialog / menu

- **Edit config**: When the overlay is up, opens this config editor.
- **Close UI element**: Closes the currently open dialog, color picker, context menu, etc.

Global mouse buttons

- **left/right/middle/back/forward**: These commands execute mouse button presses without the overlay, typically with a mouse or other pointing device still controlling cursor movement.
    - Use cases include:
        - clicking while maintaining precise cursor position/movement
        - preventing/alleviating repetitive strain injury
        - user preference over other clicking mechanisms

Free mode

- **Toggle free mode**: Toggles free mode.
- **Enter free mode**: Activates free mode.
- **Exit free mode**: Deactivates free mode.
- **Hold for free mode (beta)**: Free mode is active while the assigned key is held.
- **Hold for speed increase**: Multiples the scroll speed by the `wheel speed multiplier` value. The more keys assigned to this and held, the greater the increase.
- **Hold for speed decrease**: Divides the scroll speed by the `wheel speed multiplier` value. The more keys assigned to this and held, the greater the decrease.
- **Move up/down/left/right**: Moves the system mouse cursor.
- **Wheel up/down/left/right**: Executes smooth/continuous scrolling while held.
- **Wheel up/down/left/right fast**: Executes smooth/continuous scrolling while held, with the `wheel speed multiplier` applied.
- **Wheel step up/down/left/right**: Executes a discrete scroll event, the amount determined by the `wheel step size` setting. Responds to system 'autorepeat' when a key is held.
- **Wheel step up/down/left/right large**: Executes a discrete scroll event, the amount determined by the `wheel step size large` setting. Responds to system 'autorepeat' when a key is held.
- **Jump to top/bottom/left/right edge (experimental)** (Mac/Windows-only): Executes a very large wheel event, to move the scrollable view to the given edge.

Misc / system

- **Toggle app enabled**: Enables/disables Mouseless entirely. While disabled, all hotkeys are disabled except this one.
- **Force overlay resize**: Forces the overlay to resize/redraw itself, for rare cases where it gets out of sync with the screen.
- **Clear keyboard state**: Resets Mouseless' internal keyboard state, e.g. if a key gets 'stuck'.

Command line & debug options

Mouseless accepts command line flags that give extra visibility into what's happening under the hood, or override how the overlay is rendered on Linux.

Running from the command line

macOS

The only recommended way to run Mouseless from a terminal is with the `open` command, e.g.:

```
open -W --stdout $(tty) --stderr $(tty) --stdin $(tty) /Applications/Mouseless.app
--args -d
```

Flags for Mouseless go after `--args`.

Security advisory

Important: If you have ever run the internal binary directly (e.g. `/Applications/Mouseless.app/Contents/MacOS/mouseless`), please ensure you have revoked the **Accessibility permission** from your terminal app, otherwise any program launched from that terminal could potentially read screen content or log keystrokes.

To check your settings:

- Go to `System Settings > Privacy & Security > Accessibility`.
- Review the list. If your terminal application (e.g., Terminal, iTerm2) is present, select it and click the minus (-) button to revoke the permission.

Windows

Using PowerShell, single-user install (the default):

```
& "$env:LOCALAPPDATA\Programs\Mouseless\mouseless.exe" -d
```

All-users install:

```
& 'C:\Program Files\Mouseless\mouseless.exe' -d
```

Linux

```
flatpak run net.sonuscape.mouseless -d
```

Flags

Flag	`debug_options` key	Description
`-d`, `--debug`	—	Enables verbose **debug output**, visible in both the log file and the terminal.
`-p`, `--print-hid-events`	—	**Prints** keyboard input and mouse output to the terminal (*not* logged to file).
`-P`, `--print-hid-events-verbose`	—	Like `-p`, but also prints high-frequency mouse move, drag, wheel, and key repeat events. Implies `-p`.
`--diagnostics`	—	Prints system graphics and display info, then exits.
`--links`	—	Prints the supported deep link urls, then exits.
`--main-machine-hash`, `--trial-machine-hash`	—	Prints the machine hash used for license / trial activation, then exits (useful in support contexts).
`--list-input-devices`	—	(Linux) Prints `/dev/input` devices with their identity and how Mouseless classifies them (grab, exclude, ignore), then exits. Useful for filling in the `excluded input devices` config field.
`--overlay-renderer cairo`	`overlay_renderer`	(Linux) Forces the cairo-backed overlay instead of the webview-backed one. Default when an NVIDIA GPU is detected.
`--use-dmabuf false`	`use_dmabuf`	(Linux) Disables DMABUF rendering for webviews. Default when an NVIDIA GPU

Flag	debug_options key	Description
		is detected.
`--use-layer-shell false`	`use_layer_shell`	(Linux) Forces wlr-layer-shell on or off on Wayland. By default it is used when the compositor supports it; `false` forces the XWayland fallback.
—	`overlay_offset`	Shifts the overlay position by a pixel amount, e.g. `{ "x": 100, "y": -50 }`.

Notes:

- `--print-hid-events` gives visibility into whether Mouseless is receiving certain keyboard events and how it's interpreting them. To prevent accidental exposure, the printing will automatically disable after 5 minutes.
- Most Linux rendering-related flags have an equivalent `debug_options` config field, so they can be set persistently without launching from a terminal.
- The cairo-backed overlay does not yet have the following features: highlight_animation, dots and dashed grid_line_styles, and always_show_subgrid.
- On macOS, `--standard` and `--compatibility` (`--compat`) select the internal operating mode. Compatibility Mode is the default on Intel Macs and macOS < 14; Standard Mode is not supported on Intel hardware. These can also be set with the `MOUSELESS_MODE` environment variable.

Debug options

The `debug_options` field at the bottom of the config editor accepts a JSON object, and persists across restarts. For example:

```
{ "overlay_renderer": "cairo", "use_dmabuf": false }
```

```
{ "overlay_offset": { "x": 100, "y": -50 } }
```

See the table above for the available keys. For when to reach for these, see Troubleshooting.

Deep links

Mouseless handles 'deep links' – app-directed urls, e.g. `mouseless://toggle-overlay`. Any app or script that can open a url can therefore trigger Mouseless commands.

This is useful for:

- Triggering Mouseless commands from a launcher (e.g. Raycast, Alfred), macro tool (e.g. Keyboard Maestro, AutoHotkey), stream deck, or window manager config
- Avoiding conflicts with global hotkey handling between Mouseless and other apps (see App conflicts)
- Scripting and automation

Commands

Currently there are eight commands (**in 1.1 preview:** eleven):

Action	Url
Toggle overlay	`mouseless://toggle-overlay`
Show overlay	`mouseless://show-overlay`
Hide overlay	`mouseless://hide-overlay`
Toggle free mode	`mouseless://toggle-free-mode`
Enter free mode	`mouseless://enter-free-mode`
Exit free mode	`mouseless://exit-free-mode`
Toggle label mode (1.1 preview)	`mouseless://toggle-label-mode`
Enter label mode (1.1 preview)	`mouseless://enter-label-mode`
Exit label mode (1.1 preview)	`mouseless://exit-label-mode`
Toggle enabled	`mouseless://toggle-enabled`
Open settings	`mouseless://settings`

Running Mouseless with the `--links` CLI flag prints this list.

Using them

Most apps that can trigger a url take one directly – look for an 'open url' action – so no terminal is involved.

Where a tool runs a shell command instead (window manager keybinds, scripts), or if you just want to check that a url works:

- macOS: `open mouseless://toggle-overlay`
- Windows: `start mouseless://toggle-overlay`
- Linux: `xdg-open mouseless://toggle-overlay`

AutoHotkey has its own: `Run "mouseless://toggle-overlay"`

Behavior when app is disabled

In 1.1 preview: if the app is in the "disabled" state (e.g. you triggered the `toggle app enabled` action, or `toggle-enabled` deep link), any deep link that would activate a Mouseless mode will be ignored – e.g. `show-overlay`, `enter-free-mode`, `enter-label-mode`, or a mode toggle when that mode is off.

Callbacks

Callbacks will be added soon so Mouseless can notify other apps when a mode is exited or a command is finished.

App conflicts & compatibility

General hotkey conflicts (blocks or triggers)

If Mouseless blocks or triggers hotkeys for another app, or vice versa, changing the order they are launched in may help, or simply adjusting the keybindings in one app or the other.

On macOS, changing the `event tap location` may also alleviate these conflicts. It's in the `System / debug` section at the bottom of the config editor (**in 1.1 preview:** on the `System / Debug` page).

Workarounds for particular apps are in App-specific solutions below.

Programmatic invocation

Rather than resolving a hotkey conflict, you can sidestep it: Mouseless commands can be triggered by url instead of by hotkey, from any app or script that can open one. See Deep links.

Unexpected triggering of `tap` commands

In some cases, Mouseless `tap` commands (e.g. show overlay) are triggered because another app's hotkey blocks certain (but not all) keystrokes from reaching Mouseless.

If you experience such issues, **lowering the** `tap_threshold` setting might help prevent false triggers.

Karabiner example / explanation:

- `alt+F` is assigned an action in Karabiner
- `Alt tap` is assigned to `show overlay` in Mouseless
- User presses `alt+F` quickly
- Karabiner allows the `alt` press and release through, but blocks the `F` press and release

- Mouseless sees an `Alt tap` (because it has no visibility of the `F` key events) and displays the overlay

---

Linux input mechanics

On Wayland, Mouseless uses evdev to exclusively grab the keyboard devices, and re-emits any events that shouldn't be suppressed through a uinput virtual device.

If another tool already has an exclusive grab on all of your keyboards, Mouseless will show a "Keyboard Capture Failed" dialog listing the process(es) holding the devices. Launching Mouseless *before* that tool (or stopping the tool first) usually resolves this.

Exclude Mouseless virtual devices from other tools

All of Mouseless's virtual devices carry the Sonuscape USB vendor id `0x736e` so other input tools can identify them.

Exclude devices (virtual or real) from Mouseless

To prevent Mouseless from grabbing devices, add them to the `excluded input devices` field in the settings window (under keyboard layout in grid options; **in 1.1 preview:** at the top of the `Overlay Grid` page). Run Mouseless with the `--list-input-devices` CLI flag to view device names/ids.

App-specific solutions

AutoHotkey

To use a Windows key `tap` without triggering the start menu, use this: `~LWin::vkE8` (see reddit thread)

BetterTouchTool

If you're having issues with BTT hotkeys erroneously triggering the Mouseless overlay, a user has kindly provided this config / example, which may help you keep the same BTT hotkey, but without triggering Mouseless.

keyd

You must tell keyd to **not** grab Mouseless's virtual device. Otherwise a feedback loop is formed, and **the keyboard becomes unresponsive** until one of them is killed. Exclude the Sonuscape vendor prefix in your keyd config:

```
[ids]

*
# Exclude Mouseless virtual devices via vendor prefix
-736e

[main]
...
```

Troubleshooting

This page covers some general and platform-specific problems. Also see:

- App conflicts & compatibility – if another utility app is blocking or triggering hotkeys
- Command line & debug options – for debug output and rendering overrides

Reducing overlay lag

If the overlay feels sluggish, try some or all of these settings to make it more responsive:

```
highlight animation (ms): 0
always show subgrid: off
grid line style: lines
text shadow opacity: 0
move system cursor with virtual: off
```

If you are on Linux, you can also try changing the `overlay_renderer` (see Command line & debug options).

macOS

Startup issues: No windows or status icon visible, or hotkeys don't work

If you ever encounter these startup issues, these steps have proven to help in most cases:

1. Exit Mouseless (via Activity Monitor if status icon is not visible)
2. Go to System Settings > Privacy and Security > Accessibility
3. Delete the Accessibility permission for Mouseless, if present
    - make *SURE* Mouseless isn't running while you do this – MacOS tends to lock you out of keyboard and mouse input if you delete this permission from a running process / active event tap
    - select Mouseless and click the `-` button
4. Re-add the permission
    - Click the `+` button, add `/Applications/Mouseless.app`
    - Make sure Mouseless is toggled ON
5. Restart Mouseless

Hotkeys stop working during app usage

This may happen due to MacOS getting stuck in 'Secure Input' mode. Check the Mouseless icon in the status bar – if there's a lock icon on it, the system is in secure input mode.

Espanso has a good explainer on this issue, as well as some potential workarounds to get un-stuck (the last resort is to log out your MacOS user and log back in).

Password fields

The overlay (and keybindings involving printable characters generally) will not respond when in a password field or other field marked as sensitive. This is not a bug, but a result of MacOS's 'Secure Input' mode, which prevents applications from reading keyboard input while in these fields. To continue using Mouseless, simply leave the field (e.g. by pressing `Tab`).

Windows

Overlay appears behind Taskbar, Task Manager, etc.

For Mouseless to show the overlay above Start Menu, task manager, etc., you must:

- choose to install for 'all users'
- install to a privileged location like `Program Files` (default)

This requires admin privileges during the installation process only, not when actually running Mouseless.

The installer provides an option to uninstall other/prior installations at different locations, checked by default. Your user data is not affected by the uninstallation.

For the portable version or non-privileged install locations, Mouseless makes a best effort to show above the taskbar, but is not guaranteed to work in all situations.

WebView2 Runtime

Older versions of Windows may need the WebView2 runtime added. The Mouseless installer installs it if it's missing, which requires internet access.

On a machine without internet access, either:

- Install the runtime with Microsoft's "Evergreen Standalone Installer", from the link above, **OR**

- **In 1.1 preview:** put a Fixed Version runtime beside Mouseless:

  1. Download the x64 Fixed Version package (a `.cab` file) from the link above
  2. Extract it, e.g. `expand Microsoft.WebView2.FixedVersionRuntime.<version>.x64.cab -F:* C:\temp`
  3. Rename the extracted folder (the one containing `msedgewebview2.exe`) to `WebView2Runtime`
  4. Move it into the folder containing `mouseless.exe`, so that `msedgewebview2.exe` ends up at e.g.:
     - `C:\Program Files\Mouseless\WebView2Runtime\msedgewebview2.exe` (installed for all users)
     - `mouseless\WebView2Runtime\msedgewebview2.exe` in the extracted folder (portable)

  Mouseless uses a runtime in that folder in place of an installed one. If the `WEBVIEW2_BROWSER_EXECUTABLE_FOLDER` environment variable is set, the runtime it points to is used instead.

Linux

Overlay / Mouse Position & Scale

Mouseless does its best to automatically account for system display scaling to determine overlay size/position and mouse locations. Some users however, may need to do make manual adjustments.

At the bottom of the config editor (**in 1.1 preview:** on the `System / Debug` page) you'll find these settings, and you can hover over their labels for full explanation of possible values.

- `Overlay scaling` for fixing incorrectly sized or positioned overlays
- `Mouse scaling` for fixing incorrect mouse click locations
- `Mouse screen offset scaling` for fixing incorrect mouse click locations on multi-monitor setups (use 'Mouse scaling' for the leftmost monitor and this setting for the rest if needed)

Further, if you find that the overlay position needs manual adjustment, you can set the `overlay_offset` debug option.

Overlay / Window rendering issues

Some graphics cards / environments have issues rendering webviews with transparency, or at all.

If you encounter issues with **transparency**, set `overlay_renderer` to `cairo` to force the cairo-backed overlay (default when NVIDIA GPU is detected).

If windows appear **blank** and the overlay seems to not show at all, set `use_dmabuf` to `false` to force DMABUF rendering off for webviews (default when NVIDIA GPU is detected).

Both can be set as a command line flag or a config field, and both are described in Command line & debug options.

Overlay fades/animates in slowly (Wayland)

On Wayland compositors that support wlr-layer-shell, the overlay is a layer-shell surface with the namespace `mouseless-overlay`. If your compositor animates layer surfaces, disable animations for that namespace so the overlay appears instantly. E.g. on Hyprland:

```
layerrule = noanim, mouseless-overlay
```

Offline licensing

Network-less activation and refresh is available for air-gapped or network-restricted machines.

It requires a **lifetime** license – it will not work with a trial or subscription license.

The process:

- *From the computer you want to activate* (important), choose `Offline Licensing` when first starting the app, or `Tools` > `Offline licensing` from the Mouseless menu
- Click `Save Request File` and save the file to disk
- On a computer with network access, upload that file to the online tool
  - Enter your license key if prompted
- Click `Generate file` and save the license file
  - If you entered a key, the tool offers a re-usable request file so you don't have to enter it when refreshing
- Back in the `Offline Licensing` screen in Mouseless, click `Load License File` and choose the generated license file

Limits:

- If activating would put your license above the 5-machine limit, the online tool lists your machines and lets you replace one.
- A machine can be replaced once every 30 days.

Refresh:

Currently, you must refresh the license every 30 days, but this period will be extended or eliminated soon.

Enabling keyboard/mouse access on Wayland

This page is for Linux users on a Wayland compositor. If you're unsure if you're on Wayland or X11, run `echo $XDG_SESSION_TYPE` in a terminal.

⚠️ Security Note / Context

Wayland's security model prevents apps from having low-level access to keyboard and mouse devices. However, Mouseless requires this low-level access to: utilize global hotkeys; suppress input to apps without stealing focus; and generate mouse events.

This means you must disable Wayland's extra restrictions around keyboard/mouse access for Mouseless to work, **i.e. adopt a security model equivalent to X11's**. This **affects all programs run by your user**, not just Mouseless, though there are some possible alternatives for advanced users mentioned at the bottom.

> **Disclaimer:** The developers of Mouseless are not responsible for any issues that may arise from changing your system configuration or using elevated privileges (sudo). Do so at your own risk.

Method 1 (udev rules)

1. Create a 'mouseless' group; join it and 'input':

```
sudo groupadd --system mouseless
sudo usermod -aG input,mouseless $USER
```

2. Add this rule to allow virtual device creation:

```
sudo tee /etc/udev/rules.d/99-mouseless-input.rules <<EOF
# Output: Virtual device creation
KERNEL=="uinput", GROUP="mouseless", MODE:="0660"
EOF
```

3. Ensure the uinput module is loaded, and loads on boot:

```
sudo modprobe uinput
echo "uinput" | sudo tee /etc/modules-load.d/uinput.conf
```

4. Reload the system rules:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

5. **Log out** and log back in to apply the group changes.

Method 2 (fallback)

If you still see udev-related errors or 'no keyboards detected' errors when starting Mouseless, this method should work for you.

1. Revert changes from method 1:

```
sudo rm /etc/udev/rules.d/99-mouseless-input.rules
sudo rm -f /etc/modules-load.d/uinput.conf
sudo groupdel mouseless
sudo gpasswd -d $USER input
sudo udevadm control --reload-rules && sudo udevadm trigger
```

2. Create a systemd temporary file configuration:

```
# Forces current user/group ownership on input devices
sudo tee /etc/tmpfiles.d/mouseless-input.conf <<EOF
z /dev/uinput       0660 $USER $(id -gn) - -
z /dev/input/event* 0660 $USER $(id -gn) - -
EOF
```

3. Apply the permissions:

```
sudo systemd-tmpfiles --create /etc/tmpfiles.d/mouseless-input.conf
```

Hotplugging note: The changes made in method 2 persist across reboots, but if you plug in any keyboards after startup, you may need to run the `systemd-tmpfiles` command (step 3) again to apply permissions to them, then restart Mouseless.

Alternatives (advanced)

Run as root

Instead of changing Wayland's input security, you might be able to simply run Mouseless with `sudo`. The security implication here is that if a malicious process hijacks the Mouseless process, it will have root access to your system.

Create a dedicated user

You might be able to create a separate system user and assign the needed permissions, then run Mouseless as that user. However, running a GUI app as a different user on Wayland is complex and requires manual configuration to allow the app to connect to your display session.

If you figure out a reliable configuration for this, please share! I'd be happy to post community-contributed guides, even if they are specific to certain distros or compositors.

Roadmap

October 2026

- v1.1 preview release
  - Custom actions (all platforms)
    - set, cycle, or adjust any setting to any value(s) you want, with any hotkey you want
  - Label mode (vimium-style)
  - Revamped config editor, consolidated app window
  - Dark mode theme

Future

- Improved welcome / tutorial
- Comprehensive tutorial video
- Monitor selection improvements
  - Direct selection via hotkey - #324
- New wheel-related `hold for` commands (hold for wheel mode, stepped scrolling)
- Hide inactive portion of overlay - #261
- Improved preset management - #303
- Single overlay across multiple monitors
- Zoom on subgrid for readability - #53, #320
- Improved precision
  - Shift/case sensitive overlay keys
  - Additional grid level
- Preserve comments and field order in config file - #129
- Customizable config location - #110, #187
- Allow wheel movement in overlay mode - #194